

# PROCESOS DE INGENIERÍA DEL SOFTWARE 2

---

## Tema 12 - Ingeniería Inversa



UNIVERSIDAD  
DE ALMERÍA

### Grupo 6

Juan Francisco Escobosa Brocal

Abel Rodríguez Gutiérrez

Víctor Vallejo Uroz

|                                                                                                           |           |
|-----------------------------------------------------------------------------------------------------------|-----------|
| <b>1. Evolución Teórica: Diferencia entre Ingeniería Directa e Inversa. (Abstracción recurrente).....</b> | <b>3</b>  |
| 1.1. Ingeniería Directa.....                                                                              | 3         |
| 1.2. Ingeniería Inversa.....                                                                              | 3         |
| 1.3. Abstracción Recurrente.....                                                                          | 4         |
| <b>2. ISO 19506 (KDM): El Estándar Global.....</b>                                                        | <b>4</b>  |
| <b>2.1. Estandarización del Conocimiento para la Gobernanza de TI.....</b>                                | <b>5</b>  |
| <b>3. Marco Legal y Ético.....</b>                                                                        | <b>6</b>  |
| 3.1. Fundamento Legal de la Ingeniería Inversa.....                                                       | 6         |
| 3.2. El Derecho a la Interoperabilidad.....                                                               | 6         |
| 3.3. Sinergia Técnica: Art. 6 + ISO 19506 (KDM).....                                                      | 6         |
| 3.4. Impacto en la Gobernanza y Gestión de Activos.....                                                   | 6         |
| 3.5. Límites: Análisis de la Ley de Propiedad Intelectual en España frente a contratos EULA.....          | 7         |
| 3.5.1. Prevalencia de la Ley sobre el Contrato.....                                                       | 7         |
| 3.5.1. El Derecho a la Interoperabilidad.....                                                             | 7         |
| 3.5.2. Rompiendo el "Vendor Lock-in".....                                                                 | 7         |
| <b>4. Vanguardia Tecnológica.....</b>                                                                     | <b>7</b>  |
| <b>5. Casos de Estudio: Análisis técnico de Stuxnet.....</b>                                              | <b>10</b> |
| <b>5.1. Stuxnet.....</b>                                                                                  | <b>10</b> |
| 5.2. Ingeniería Inversa en hardware.....                                                                  | 10        |
| 5.3. Comparación RISC-V vs x86.....                                                                       | 11        |

# 1. Evolución Teórica: Diferencia entre Ingeniería Directa e Inversa. (Abstracción recurrente)

La comprensión de los sistemas tecnológicos modernos requiere del dominio del flujo de la información y diseño. Este flujo se mueve a través de diferentes niveles de abstracción. Según el nivel, se ha llegado a dos disciplinas fundamentales y complementarias como son la Ingeniería Directa y la Ingeniería Inversa.

## 1.1. Ingeniería Directa

La ingeniería directa, es el proceso de desarrollo más tradicional. Su origen viene de las raíces de la ingeniería clásica y la arquitectura, donde antes de construir algo se diseñan los planos. Dentro de la informática, este concepto se estructuró y estandarizó en los 70 con la introducción de los Ciclos de Vida del Desarrollo de Software(SDLC), como en el modelo en Cascada. Nació de la necesidad de aplicar rigor, previsibilidad y control de calidad a la creación de sistemas cada vez más complejos.

Para aplicar ese rigor y previsibilidad que la caracteriza, la ingeniería directa divide el trabajo en etapas lógicas y secuenciales:

- **Ingeniería de Requisitos:** Es el punto de partida. Se investiga y documenta qué debe hacer exactamente el sistema, qué problemas resuelve y qué restricciones tiene (presupuesto, tiempo, tecnología).
- **Diseño del Sistema (Los "Planos"):** Se traduce lo que el usuario necesita en esquemas técnicos. Se define la arquitectura general, el diseño de la base de datos, las interfaces de usuario y los diagramas de componentes (frecuentemente utilizando lenguaje UML).
- **Implementación (Codificación):** Es el momento en el que los programadores toman los planos y los traducen a un lenguaje de programación específico (Java, Python, C++, etc.). Es el paso definitivo de la teoría a la práctica.
- **Pruebas (Testing):** Se verifica de manera exhaustiva que el código creado cumple con los requisitos y diseños planteados en las primeras fases, garantizando el control de calidad.
- **Despliegue y Mantenimiento:** El software se entrega al cliente o se pone en producción, iniciando su etapa de vida útil.

## 1.2. Ingeniería Inversa

Por otro lado, la ingeniería inversa es un poco más moderna. Históricamente, nació en el ámbito de la ingeniería mecánica, teniendo especial impacto en la Guerra Fría donde los ejércitos capturaban armamento enemigo y lo desmontaban pieza a pieza para entender su tecnología y replicarla o contrarrestarla.

En el ámbito del software, se formalizó entre los 80 y 90 como respuesta a la necesidad de lograr interoperabilidad entre sistemas cerrados, recuperar código fuente perdido en sistemas legacy y, más actualmente el análisis de malware.

El funcionamiento de este tipo de ingeniería es totalmente opuesto a la ingeniería directa. En vez de partir de un punto abstracto y desarrollar más adelante, partimos de un sistema o software ya creado y lo vamos descomponiendo para descubrir su diseño, arquitectura, funcionamiento interno o extraer conocimiento del mismo sin tener acceso previo al código fuente o documentación original.

Tal y como se puede ver, la principal diferencia radica en la dirección del ciclo de vida, mientras en la Ingeniería Directa vamos de lo abstracto a la implementación (Idea->Código), en la Ingeniería Inversa vamos de la implementación a la abstracción (Código->Comprensión)

Debido a estas diferencias cada una se utiliza en situaciones distintas siendo que la Ingeniería Directa se usa para crear nuevos productos, funcionalidades, sistemas o estándares. Mientras que la Ingeniería Inversa se usa actualmente para auditorías de seguridad, análisis de malware o asegurar interoperabilidad.

### **1.3. Abstracción Recurrente**

La Abstracción Recurrente es el modelo conceptual que integra la Ingeniería Inversa y la Ingeniería Directa en un flujo de trabajo continuo y evolutivo.

En vez de verlas como procesos aislados, la abstracción recurrente postula que un sistema vivo requiere moverse cíclicamente por diferentes niveles de abstracción.

Este concepto se visualiza técnicamente mediante el Modelo de Herradura, que describe el camino que sigue la información:

Ascenso (Ingeniería Inversa): Se parte del código ejecutable o del hardware físico. A medida que analizamos, "subimos" niveles de abstracción: de señales eléctricas a binarios, de binarios a código fuente, y de código fuente a diagramas de arquitectura y requisitos.

Cúspide (Análisis y Transformación): En el nivel más alto de abstracción (el modelo conceptual), el ingeniero no trabaja con código, sino con lógica pura. Aquí es donde se detectan fallos, se planifican mejoras o se deciden cambios de arquitectura.

Descenso (Ingeniería Directa): Una vez transformado el modelo, se inicia el camino hacia abajo, traduciendo las nuevas especificaciones en código optimizado, parches de seguridad o nuevas versiones físicas.

Por último, cabe mencionar que se denomina recurrente porque este proceso no tiene un final definitivo. Cada nueva implementación generada mediante Ingeniería Directa eventualmente se convertirá en el objeto de estudio de una futura fase de Ingeniería Inversa.

## 2. ISO 19506 (KDM): El Estándar Global

La normativa ISO/IEC 19506 define el Knowledge Discovery Metamodel (KDM), un estándar desarrollado por el Object Management Group (OMG) para la Modernización Impulsada por la Arquitectura (ADM).

Su propósito fundamental no es la simple documentación, sino la extracción y normalización del conocimiento de sistemas de software existentes. KDM permite que herramientas de análisis automático "entiendan" la estructura, los datos y la lógica de negocio de un sistema legado, independientemente del lenguaje de programación original.

El conocimiento dentro de la ISO 19506 no se almacena de forma plana, sino que se organiza en un ecosistema jerárquico de cuatro capas. Esta estructura permite separar los detalles técnicos de la infraestructura de la lógica de negocio pura:

- Capa de Infraestructura (Infrastructure Layer): Se encarga de mapear los activos físicos y el entorno de construcción del software. Identifica y organiza archivos de código fuente, scripts de despliegue, estructuras de directorios y archivos de configuración de compilación. Es el nivel que permite responder con exactitud dónde reside físicamente cada componente del sistema.
- Capa de Programa (Program Layer): Es el núcleo del análisis técnico donde se detecta la lógica y la sintaxis. En este nivel, KDM identifica unidades lógicas como clases (ClassUnit), interfaces, métodos y tipos de datos. Además, descompone el flujo de control, permitiendo ver cómo interactúan las variables y los parámetros dentro del código.
- Capa de Recursos (Resource Layer): Gestiona todo lo que se encuentra "fuera" del código fuente pero que es vital para su funcionamiento. Aquí se registran las interacciones con bases de datos (tablas y columnas), interfaces de usuario, protocolos de red, APIs externas y eventos del sistema operativo. Es la capa clave para entender las dependencias del sistema.
- Capa de Abstracción (Abstraction Layer): Representa el nivel más alto de conocimiento, traduciendo la técnica a conceptos de negocio. Organiza los elementos en reglas de proceso y componentes arquitectónicos. Esta capa es fundamental para la gobernanza, ya que permite ver la orquestación de servicios y la estructura lógica del software sin perderse en los detalles de implementación.

### 2.1. Estandarización del Conocimiento para la Gobernanza de TI

La estandarización mediante la ISO 19506 transforma el código fuente —un dato crudo y a menudo fragmentado— en un activo de inteligencia estructurado. Para que este conocimiento sea útil en la toma de decisiones de alto nivel, el estándar opera bajo cuatro mecanismos de gobernanza:

Homogeneización Tecnológica (Neutralidad): La gobernanza de TI a menudo se ve obstaculizada por la diversidad de lenguajes (sistemas híbridos de COBOL, Java, .NET). KDM estandariza estos activos

traduciéndolos a un inventario común. Esto permite que la dirección de TI tenga una visión unificada de todo el patrimonio de software de la empresa, eliminando silos de información técnica.

**Interoperabilidad y Control de Activos (Formato XMI):** Al utilizar el formato estándar XMI (XML Metadata Interchange), el conocimiento extraído deja de depender de una herramienta o consultora específica. Esto garantiza la soberanía del dato técnico por parte de la organización, evitando el vendor lock-in y asegurando que los modelos de arquitectura sean legibles y auditables a largo plazo.

**Gestión de Riesgos mediante Análisis de Impacto:** La estandarización permite mapear de forma precisa las relaciones de dependencia entre componentes (Calls, Reads, Writes). En términos de gobernanza, esto se traduce en la capacidad de realizar análisis de impacto predictivos: antes de realizar cualquier cambio o migración, es posible identificar exactamente qué procesos de negocio y recursos se verán afectados, reduciendo drásticamente el riesgo de interrupciones operativas.

**Auditoría Objetiva de la Deuda Técnica:** La ISO 19506 proporciona un marco métrico para evaluar la calidad del software sin ambigüedades. Permite identificar automáticamente violaciones de principios de diseño (como SOLID) o estructuras excesivamente complejas. Para la gobernanza, esto permite priorizar las inversiones en mantenimiento y refactorización basadas en datos objetivos, optimizando el retorno de inversión (ROI) en la modernización de sistemas.

### 3. Marco Legal y Ético.

#### 3.1. Fundamento Legal de la Ingeniería Inversa

La **Directiva 2009/24/CE** sobre la protección jurídica de programas de ordenador constituye el pilar legal que permite los procesos de modernización de software en la Unión Europea. El Artículo 6 (Descompilación) establece una excepción crítica a los derechos de autor, permitiendo la ingeniería inversa sin autorización del titular original bajo condiciones específicas.

#### 3.2. El Derecho a la Interoperabilidad

El artículo legitima la reproducción y traducción de la forma del código cuando estos actos sean indispensables para obtener la información necesaria para lograr la **interoperabilidad** de un programa creado de forma independiente con otros programas.

En el contexto de la ISO 19506 (KDM), este marco legal garantiza que una organización tiene derecho a:

- Analizar las interfaces y la estructura de su software legado.
- Extraer la lógica de negocio para permitir la comunicación con arquitecturas modernas (Cloud, Microservicios).
- Documentar sistemas donde la información técnica original ha sido retenida o perdida por el proveedor.

#### 3.3. Sinergia Técnica: Art. 6 + ISO 19506 (KDM)

La relación entre el marco legal y el estándar técnico es el motor de la soberanía tecnológica:

- El Salvoconducto Legal (Art. 6): Proporciona el permiso para "abrir la caja negra" del software sin infringir la ley, siempre que el fin último sea la conexión y evolución del ecosistema digital.
- La Ejecución Técnica (KDM): Proporciona el método estandarizado para registrar esa información. Mientras el Art. 6 autoriza el proceso, la ISO 19506 asegura que el resultado sea un activo de conocimiento útil, auditable e independiente de cualquier herramienta específica.

### 3.4. Impacto en la Gobernanza y Gestión de Activos

La inclusión de esta directiva en la estrategia de TI elimina el **Vendor Lock-in** (secuestro tecnológico). Desde una perspectiva de gobernanza, el Artículo 6 asegura que:

1. Continuidad del Negocio: La empresa no queda cautiva de un proveedor que se niega a proporcionar documentación o APIs.
2. Transparencia de Activos: Se puede auditar la lógica de sistemas críticos para garantizar que cumplen con normativas de seguridad y escalabilidad.
3. **Reducción de Riesgos:** El conocimiento extraído permite una transición segura hacia nuevos lenguajes y plataformas, protegiendo la inversión histórica en lógica de negocio.

### 3.5. Límites: Análisis de la Ley de Propiedad Intelectual en España frente a contratos EULA.

En el marco jurídico español, existe una tensión constante entre las restricciones de los contratos de licencia (EULA) y los derechos del usuario. Sin embargo, la Ley de Propiedad Intelectual (Art. 100) establece límites claros que favorecen la soberanía tecnológica del cliente.

#### 3.5.1. Prevalencia de la Ley sobre el Contrato

Aunque un contrato EULA prohíba explícitamente la ingeniería inversa, en España esta cláusula es nula de pleno derecho si el objetivo es lograr la interoperabilidad. La LPI garantiza que el usuario legítimo pueda analizar, estudiar o probar el funcionamiento del programa para conectar sistemas, independientemente de lo que diga la licencia del fabricante.

#### 3.5.1. El Derecho a la Interoperabilidad

La estandarización mediante ISO 19506 (KDM) se apoya en este derecho legal. Se permite la descompilación y el análisis del código cuando:

- La información es necesaria: El proveedor no ha facilitado documentación técnica o APIs para la conexión.
- Fin específico: El proceso busca exclusivamente que el software antiguo se comunice con programas nuevos (Nube, Microservicios).
- Ámbito limitado: Solo se analiza la parte del código indispensable para dicha conexión.

#### 3.5.2. Rompiendo el "Vendor Lock-in"

Desde la gobernanza de TI, este marco legal asegura que la empresa no sea "rehén" de su proveedor. La LPI protege el derecho de la organización a extraer su propia lógica de negocio y datos para evitar el secuestro tecnológico, permitiendo una evolución hacia arquitecturas más modernas y eficientes.

Conclusión Estratégica: El miedo legal a los contratos EULA no debe frenar la modernización. La normativa española blinda la ingeniería inversa siempre que su fin sea la interoperabilidad y la supervivencia técnica del sistema, no la copia del producto original.

## **4. Vanguardia Tecnológica.**

La aplicación práctica de la ingeniería inversa, amparada por normativas como la Directiva 2009/24/CE y estructurada mediante el estándar ISO/IEC 19506 (KDM), requiere de un marco instrumental capaz de procesar ejecutables compilados y elevar su nivel de abstracción. Históricamente, este proceso se ha visto limitado por barreras tecnológicas que actualmente están siendo superadas mediante la estandarización de herramientas y la implementación de modelos de inteligencia artificial.

### **4.1. Estandarización de la Infraestructura de Análisis**

Durante décadas, la auditoría de código binario dependió de herramientas privativas caracterizadas por altos costes de licenciamiento y arquitecturas cerradas (caja negra). Este modelo limitaba la auditabilidad del propio proceso de ingeniería inversa y fomentaba el *vendor lock-in* en los procedimientos forenses y de recuperación de activos.

El paradigma sufrió una disrupción metodológica significativa con la desclasificación y liberación de Ghidra por parte de la Agencia de Seguridad Nacional de los Estados Unidos (NSA). La transición hacia un modelo de código abierto (*Open Source*) no solo democratizó el acceso a capacidades de desensamblado de grado militar, sino que estableció un marco de trabajo estandarizado y extensible. Desde una perspectiva académica y operativa, esta apertura permite a la comunidad científica y a las organizaciones auditar los algoritmos de descompilación, desarrollar *plugins* adaptados a arquitecturas específicas y facilitar el análisis colaborativo y concurrente sobre un mismo binario. En consonancia con los principios del estándar KDM, esta estandarización garantiza que la extracción de conocimiento sea independiente del proveedor y altamente interoperable.

### **4.2. Arquitectura de Descompilación y Transformación.**

El núcleo técnico del análisis estático moderno reside en la capacidad de transformar instrucciones de máquina (específicas de un hardware determinado) en representaciones de alto nivel, independientes de la arquitectura original. Herramientas modulares como Ghidra logran esto mediante una arquitectura de traducción en múltiples fases.

En la primera fase, el motor de desensamblado procesa el código binario y lo traduce a una Representación Intermedia (IR), denominada *P-Code* en el ecosistema de Ghidra. El P-Code actúa como un lenguaje universal que normaliza las operaciones, abstrayendo las particularidades de procesadores como x86, ARM o MIPS.



Una vez normalizado el código, el descompilador aplica algoritmos de análisis de flujo de datos y análisis de propagación de constantes para simplificar las operaciones. Posteriormente, se genera un Grafo de Flujo de Control (CFG, por sus siglas en inglés), una estructura de datos matemática donde los nodos representan bloques básicos de instrucciones sin saltos, y las aristas (flechas dirigidas) representan las bifurcaciones lógicas y transiciones de estado del programa (condicionales, bucles).

Finalmente, la arquitectura mapea este grafo sobre Árboles de Sintaxis Abstracta (AST) para emitir un pseudocódigo estructurado, generalmente con una sintaxis análoga a C. Este proceso inverso —que asciende desde el nivel físico hasta la Capa de Programa definida en el estándar KDM— permite al ingeniero operar sobre estructuras de control lógicas (bloques if, bucles while) en lugar de sobre registros de memoria y saltos incondicionales (JMP).

### 4.3. Pérdida Semántica y Carga Cognitiva en la Evaluación Heurística.

A pesar de la sofisticación de la descompilación a representaciones intermedias, el análisis estático se enfrenta a un desafío matemático y lingüístico insoslayable: la pérdida irreversible de metadatos durante el proceso de compilación original. Un binario despojado de símbolos de depuración carece de estructuras semánticas humanas; los identificadores de variables, los nombres de funciones y los comentarios son sustituidos por direcciones de memoria y registros de CPU.

Este fenómeno genera un "abismo semántico" entre la lógica de máquina y la lógica de negocio. Para aislar la capa de aplicación real y sortear el código de inicialización inyectado por el compilador (*compiler stubs*), el analista debe recurrir a heurísticas de trazado. Una de las metodologías más eficaces es la resolución de referencias cruzadas (XREFs) a partir de cadenas de texto estáticas definidas en memoria. Mediante la localización de literales específicos, es posible trazar el flujo de ejecución hacia atrás hasta llegar al núcleo del algoritmo.

No obstante, la reconstrucción manual de la intención del programador —conocida como abstracción recurrente semántica, donde se renombran variables genéricas basándose en su uso contextual— impone una carga cognitiva masiva. Este proceso de inferencia manual representa el principal cuello de botella en la auditoría de sistemas a gran escala.

### 4.4. Integración de Modelos de Lenguaje (LLM) para la Inferencia de Contexto.

Para resolver la ineficiencia de la inferencia manual, el estado del arte metodológico ha evolucionado hacia la integración de Modelos de Lenguaje Grande (LLMs) directamente en los motores de descompilación. A diferencia del análisis heurístico tradicional, que depende de firmas predefinidas, los LLMs aplican análisis probabilístico sobre el Árbol de Sintaxis Abstracta (AST) generado por herramientas como Ghidra.

Estos modelos son capaces de evaluar el contexto de una variable genérica —observando a qué funciones de la API del sistema operativo se transfiere como parámetro o qué operaciones aritméticas sufre— para inferir y sugerir identificadores semánticamente precisos. Asimismo, los algoritmos de IA permiten condensar subgrafos complejos del Grafo de Flujo de Control en resúmenes funcionales en lenguaje natural. Esta automatización transforma la extracción de conocimiento, reduciendo drásticamente la latencia en la transición desde la Capa de Programa hacia la Capa de Abstracción definida en el estándar KDM.

### 4.5. Modificación Estructural.

La culminación del proceso de ingeniería inversa a menudo requiere la validación de hipótesis o la neutralización de comportamientos anómalos (o mecanismos de ofuscación) directamente sobre el ejecutable. Una vez que la lógica de dominio ha sido aislada y comprendida semánticamente, se procede a la manipulación del código a nivel de instrucción, técnica conocida como *binary patching*.

Esta fase técnica implica la alteración de los nodos condicionales identificados en el Grafo de Flujo de Control. Al invertir a nivel de ensamblador los mnemónicos de evaluación condicional (por ejemplo,

mutando una instrucción de salto si es cero, JZ, por un salto si no es cero, JNZ), se fuerza un desvío determinista en el flujo de ejecución original. Esta técnica no solo es fundamental para la corrección de vulnerabilidades en sistemas *legacy* carentes de código fuente, sino que constituye el mecanismo principal para habilitar la interoperabilidad de componentes cerrados, cumpliendo así con las excepciones legales de descompilación contempladas en normativas internacionales.

## 5. Casos de Estudio: Análisis técnico de Stuxnet

### 5.1. Stuxnet

La ingeniería inversa vio un despunte en su aplicación práctica, alcanzando su máxima complejidad cuando se empezó a enfrentar a ciberarmas y a la estructura física de los procesadores.

La primera de estas ciberarmas analizada con Ingeniería inversa fue Stuxnet en 2010. Este fue considerado el primer ciberarma del mundo diseñado para causar un daño físico real. Su objetivo era sabotear el programa nuclear de Irán, atacando centrifugadoras en la planta de Natanz, alterando los controladores lógicos de Siemens, destruyendo alrededor de 1000 máquinas.

A grandes rasgos, infectaba equipos con windows mediante memorias USB y buscaba los controladores para reprogramar las máquinas haciendo que aumenten su velocidad romperlas mientras se enviaban señales falsas a los operadores para que piensen que todo funcionaba con normalidad.

Este ciberarma se rumorea que fue creado por EEUU e Israel en conjunto.

Con el tiempo y la aplicación de Ingeniería Inversa, se consiguió analizar el código malicioso descubriendo que este tenía una de las estructuras más sofisticadas hasta la fecha dividida en varias capas de complejidad.

El primer descubrimiento de los Ingenieros fue que Stuxnet explotaba 4 vulnerabilidades de día cero en Windows. La más famosa de esas cuatro fue la LVK(CVE-2010-2568), que dejaba ejecutar código de manera arbitraria con solo visualizar el icono de un USB infectado en el explorador de Windows.

Además, para eludir defensas de los sistemas, al desensamblar el código se descubrió que los drivers maliciosos estaban firmados con certificados digitales legítimos que habían robado a empresas tecnológicas reales como Realtek.

Por último, el hito más crítico del análisis fue aislar y realizar ingeniería inversa al código de los Controladores Lógicos Programables Siemens S7-315. Stuxnet interceptaba la comunicación entre el software de monitorización y el hardware. Modificaba la frecuencia de los variadores de los motores mientras enviaba datos falsos pregrabados simulando un funcionamiento normal.

## 5.2. Ingeniería Inversa en hardware

Mientras que Stuxnet fue un caso de software que atacaba hardware, la ingeniería inversa también se puede realizar directamente sobre hardware lo que implica deconstruir los propios microchips y PCBs. Para ello, se suelen aplicar las siguientes técnicas:

- Decapsulación (De-capping): Uso de ácidos corrosivos (como el ácido nítrico) para disolver la cubierta epoxi del chip sin dañar el silicio subyacente.
- Microscopía Electrónica (SEM y TEM): Mapeo fotográfico de las capas de metal y polisilicio a nivel nanométrico para reconstruir las puertas lógicas y extraer ROMs "fotografiando" los bits físicos (unos y ceros).
- Modificación con Haz de Iones Enfocados (FIB): Permite a los analistas cortar pistas o depositar metal nuevo a escala nanométrica para "recablear" el chip, saltándose comprobaciones de seguridad físicas o extrayendo claves criptográficas secretas.
- Ataques de Canal Lateral (Side-Channel Attacks): Análisis no destructivo donde se infiere el comportamiento interno o las claves criptográficas midiendo el consumo de energía (DPA) o las emisiones electromagnéticas durante el procesamiento de datos.

## 5.3. Comparación RISC-V vs x86

La Arquitectura del Conjunto de Instrucciones (ISA, por sus siglas en inglés) define cómo el software se comunica con el hardware. La elección del ISA tiene un impacto drástico en cómo se realiza la ingeniería inversa y el análisis de seguridad de un procesador.

### 5.3.1. Arquitectura x86 (Intel / AMD)

Paradigma CISC (Ordenador con Conjunto de Instrucciones Complejas): Construido con un conjunto de instrucciones extenso y complejo. Una instrucción de software puede ejecutar varias operaciones de bajo nivel (carga, operar, almacenar).

Propietaria de Caja Negra: Es un estándar cerrado, con estricta propiedad intelectual. El funcionamiento preciso de la CPU es secreto.

El problema del microcódigo: Las instrucciones x86 son tan complejas que el procesador no puede ejecutarlas directamente en hardware, por lo que las traduce internamente con una capa de "microcódigo" (un firmware oculto y encriptado dentro de la CPU).

Ingeniería Inversa: Trabajar con el análisis de x86 es "a ciegas". Los analistas a menudo encuentran instrucciones que no están documentadas o "puertas traseras" ocultas (como el subsistema de gestión

Intel - Ring -3) que operan fuera del control del sistema operativo. La complejidad CISC y la falta de transparencia obligan a que la auditoría de seguridad sea un proceso iterativo de prueba y error.

### **5.3.2. Arquitectura RISC-V**

Paradigma RISC (Reduced Instruction Set Computer): Posee un conjunto de instrucciones muy reducido, sencillas y altamente optimizadas. Cada instrucción hace sólo una cosa y se ejecuta, por lo general, en un sólo ciclo de reloj.

Caja Blanca Abierta (Open Standard): El ISA es de código abierto, gratuito y de acceso público. Cualquier individuo o compañía puede diseñar, crear o alterar un chip RISC-V sin tener que pagar regalías.

Diseño Modular: Está formado por un conjunto base obligatorio (muy pequeño) y extensiones estándar opcionales (de multiplicación, coma flotante, criptografía). No tiene el legado (código de décadas pasadas) que sí tiene x86.

Ingeniería Inversa (Transparencia y Verificabilidad): Puesto que el ISA es de dominio público, la barrera para entender “qué debería hacer el chip” se elimina. En las implementaciones de código abierto (donde el código RTL del procesador es público), la ingeniería inversa tradicional se convierte en una Auditoría de Caja Blanca. Los analistas de seguridad pueden verificar de forma matemática que el diseño del hardware cumple con las especificaciones y que no contiene troyanos de hardware, antes de que se fabrique el chip (validación antes del silicio o “pre-silicon validation”).